NEXUS — Manual técnico y preparación para el examen H3

Programación II · Proyecto NexusTrainingSystem (C#, aplicación de consola)

0. Cómo leer este manual

Todo el proyecto vive en **un solo archivo** (lo llamo `Program.cs`, namespace `NexusTrainingSystem`; si el tuyo tiene otro nombre, usa el real). Por eso cada ejemplo se identifica con **clase + método**: puedes comprobarlo buscando ese nombre en tu código.

Marcas que uso en todo el documento:

- **[NEXUS]**: el código existe tal cual en tu proyecto.
- **[COMPLEMENTARIO]**: ejemplo que escribí yo porque NEXUS no implementa ese concepto. No está en tu proyecto; si lo usas en el examen, preséntalo como un ejemplo propio.

Mapa rápido: qué concepto aparece y dónde

Concepto	¿Está en NEXUS?	Dónde verlo
Clases, objetos, instanciación	Sí	`RealidadDimensional`, `Explorador`, `Robot`, `Enemigo`, `Objeto`
Constructores	Sí	`RealidadDimensional(...)`, `Explorador(...)`, `Mision(...)`
Sobrecarga de constructores	**No** (hay parámetros opcionales y `params`)	Complementario en 2.3
Encapsulamiento	Sí	`RealidadDimensional`, `Personaje`, `Explorador`
`this`	Sí	Constructor y `AjustarInestabilidad` de `RealidadDimensional`
`base`	Sí	`Cadete.RecibirDano` y los constructores de las derivadas
Herencia	Sí	`Personaje` → `Cadete` → `Explorador`; `Robot`; `Enemigo`
`virtual` / `override`	Sí	`Personaje.RecibirDano`
`abstract`	Sí	`Personaje` (`Describir`, `Reaccionar`)
Polimorfismo	Sí	`Program.Encuentro`
Interfaces	**No**	Complementario en la sección III
Composición	Sí	`Cadete` → `Inventario` → `List<Objeto>`; `Mision` → `List<Objetivo>`
`List<T>`	Sí	`Inventario`, `Mision`, `Program.catalogoRealidades`
Lambdas	Sí	`Inventario.Total`, `Mision.Completada`
LINQ `Sum`, `FirstOrDefault`, `Any`, `All`, `Take`, `Count`	Sí	`Inventario`, `Mision`, `Program.MostrarMenu`
LINQ `Where`, `Select`, `OrderBy`	**No**	Complementario en la sección IV
Extras: eventos, excepción propia, hilos y sockets	Sí	`RealidadDimensional`, `Explorador`, `IrisConsola`

I. Arquitectura y programación orientada a objetos

1.1 Propósito y funcionamiento general de NEXUS

NEXUS es un **simulador de entrenamiento por consola**. El jugador es un *Explorador* que se conecta a realidades dimensionales, las explora, combate entidades, administra un inventario y cumple una misión. El flujo completo está en `Program.Main`:

1. **Registro**: pide nombre, edad, realidad asignada, energía y estabilidad iniciales (`LeerTexto`, `LeerEnteroEnRango`).
2. **Creación de objetos**: crea tres `RealidadDimensional`, las guarda en `catalogoRealidades`, crea el `Explorador` y le conecta los **eventos** (`AnomaliaDetectada`, `ColapsoDetectado`).
3. **Autorización** (`EvaluarAutorizacion`): edad ≥ 15, energía ≥ 30 y estabilidad ≥ 40. Si algo falla, el programa termina.
4. **Misión** (`IniciarJuego`): entrega un inventario inicial (célula, escudo, analizador) y la misión *La anomalía perdida*, con 4 objetivos.
5. **Ciclo principal**: `while (explorador.Conectado)` muestra el menú (9 operaciones), ejecuta la elegida y, al final de cada vuelta, revisa tres reglas: resonancia estable, condición crítica y colapso dimensional.
6. **Fin**: se sale por desconexión voluntaria, muerte del Explorador, 3 alertas IRIS o un segundo colapso. `FinalizarJuego` imprime el resumen.

Además, `IrisConsola` abre un servidor TCP en `localhost:5050`, se conecta a sí mismo y un hilo en segundo plano imprime los mensajes recibidos como `[CONSOLA IRIS]`. Es solo un canal de notificaciones: si no se puede abrir, el juego continúa.

1.2 Estructura del proyecto y responsabilidades

Clase / tipo	Naturaleza	Responsabilidad
`Azar`	Clase estática	Entrega un único `Random` compartido (`Azar.Rng`).
`OperacionInvalidaException`	Excepción propia (hereda de `Exception`)	Señalar errores de uso: menú, inventario, combate.
`EstadoRealidad`	`enum`	Estable, Inestable, Critica.
`RealidadDimensional`	Clase	Modela una realidad: inestabilidad, estado calculado, costos; avisa con un evento cuando cambia de estado.
`IrisConsola`	Clase	Notificaciones por sockets TCP y un hilo de escucha.
`Personaje`	Clase **abstracta**	Base común: vida, ataque, daño, curación. Define el contrato `Describir` y `Reaccionar`.
`Cadete`	Derivada de `Personaje`	Añade XP, nivel, escudo, bono de ataque e `Inventario`.
`Explorador`	Derivada de `Cadete`	El personaje del jugador: energía, estabilidad, peligro, fragmentos, realidad actual y eventos.
`Robot`, `Enemigo`	Derivadas de `Personaje`	Entidades hostiles con reglas propias de daño y de reacción.
`TipoObjeto`	`enum`	Celula, Escudo, Modulo, Analizador.
`Objeto`	Clase	Un ítem (nombre, tipo, icono, cantidad). Fábricas estáticas `Crear`, `ModuloDesconocido`, `Aleatorio`.
`Inventario`	Clase	Colección con capacidad 10: `Agregar`, `Consultar`, `Buscar`, `Utilizar`, `Eliminar`.

Clase / tipo	Naturaleza	Responsabilidad
`Objetivo`, `Mision`	Clases	Los objetivos y la regla «la misión está completa si todos están completados».
`Program`	Clase con solo miembros `static`	Interfaz de consola, mapas, encuentros, reglas de eventos y flujo general.

Observaciones de diseño (útiles para argumentar):

- Las reglas del dominio están bien repartidas en clases con responsabilidad propia (`RealidadDimensional` decide su estado, `Inventario` decide si cabe un objeto, `Mision` decide si está completa).
- `Inventario.Consultar` y `Mision.Mostrar` escriben en la consola: mezclan modelo y presentación.
- `Program` concentra demasiada lógica (mapas, combate, eventos, validación de entrada).

1.3 Diagrama de clases y relaciones entre objetos

Jerarquía de herencia [NEXUS]:

```
Personaje (abstract)
   +Nombre, VidaMax, Vida, Ataque, Analizado, EstaVivo
   +virtual RecibirDano(int)   +Curar(int)
   +abstract Describir()       +abstract Reaccionar(Explorador)
├── Cadete        override RecibirDano, Describir, Reaccionar
│   └── Explorador   (no sobrescribe nada; añade Energia, Estabilidad, eventos)
├── Robot         override RecibirDano, Describir, Reaccionar
└── Enemigo       override RecibirDano, Describir, Reaccionar
```

Composición, asociación y dependencias [NEXUS]:

```
Program (miembros static)
  ├─ iris               : IrisConsola
  ├─ catalogoRealidades : List<RealidadDimensional>
  ├─ misionActual       : Mision  ◆── Objetivos : List<Objetivo>
  └─ entidadActual      : Personaje  (en la práctica un Robot o un Enemigo)

Explorador ──hereda──► Cadete ◆── Inventario ◆── List<Objeto>
Explorador ──────────► RealidadDimensional   (asociación: es una del catálogo)
```

Relación	Dónde ocurre	Tipo
`Cadete`, `Robot`, `Enemigo` → `Personaje`	declaración `: Personaje`	Herencia («es un»)
`Explorador` → `Cadete`	declaración `: Cadete`	Herencia (segundo nivel)
`Cadete` → `Inventario`	`public Inventario Inventario { get; } = new Inventario();`	Composición: el inventario nace y muere con el cadete
`Inventario` → `Objeto`	`private readonly List<Objeto> objetos`	Composición con colección
`Mision` → `Objetivo`	el constructor hace `new Objetivo(o)`	Composición con colección
`Explorador` → `RealidadDimensional`	recibida en el constructor, también guardada en `catalogoRealidades`	Asociación/agregación: la realidad existe sin el Explorador
`Robot.Reaccionar`, `Enemigo.Reaccionar`, `Inventario.Utilizar` → `Explorador`	parámetro del método	Dependencia (solo lo usan, no lo guardan)
`RealidadDimensional.EstadoCambiado` → `Program.ManejarCambioEstadoRealidad`	`r.EstadoCambiado += ...` en `Main`	Evento (patrón observador)

Relación	Dónde ocurre	Tipo
`Explorador.AnomaliaDetectada`, `ColapsoDetectado` → `ManejarAlertaIris`, `ManejarColapsoDimensional`	`explorador.AnomaliaDetectada += ...`	Evento

Un detalle de diseño digno de mención: `Personaje.Reaccionar(Explorador cadete)` obliga a la clase base a conocer a una de sus derivadas (`Explorador`). Es una dependencia circular de tipos; sería más limpio recibir un `Cadete` o una interfaz (ver la sección III).

1.4 Programación estructurada frente a programación orientada a objetos

Aspecto	Estructurada	Orientada a objetos
Unidad básica	Funciones y procedimientos	Objetos (datos + comportamiento)
Datos	Variables globales o parámetros sueltos	Estado encapsulado dentro del objeto
Añadir un caso nuevo	Modificar los `if`/`switch` en varias funciones	Crear una clase derivada
Reutilización	Copiar o parametrizar funciones	Herencia, composición, interfaces
En NEXUS	`Program`: métodos `static`, estado en campos estáticos (`misionActual`, `entidadActual`, `advertenciasCriticas`), mapas como matriz `char[,]`	`Personaje` y derivadas, `RealidadDimensional`, `Inventario`, `Mision`

NEXUS es un diseño híbrido. Para el mismo problema, el daño que recibe una entidad, compara los dos estilos.

Estilo estructurado [COMPLEMENTARIO]:

```
static int DanoRecibido(string tipo, int dano, bool analizado)
{
    if (tipo == "Robot" && !analizado) return dano * 6 / 10;
    if (tipo == "Enemigo" && analizado) return dano * 3 / 2;
    return dano;
}
// Un tipo nuevo (por ejemplo "Jefe") obliga a editar esta función
// y todas las demás que tengan un if sobre el tipo.
```

Estilo orientado a objetos [NEXUS] (`Robot.RecibirDano` y `Enemigo.RecibirDano`):

```
// Robot
public override int RecibirDano(int dano)
{
    if (!Analizado) dano = dano * 6 / 10;
    return base.RecibirDano(dano);
}
// Enemigo
public override int RecibirDano(int dano)
{
    if (Analizado) dano = dano * 3 / 2;
    return base.RecibirDano(dano);
}
```

Cada clase conoce **su propia regla**; quien ataca solo escribe `ent.RecibirDano(dano)`. Un tipo nuevo se agrega sin tocar el código existente.

Dónde NEXUS rompe su propio diseño OO: en `Program.Encuentro` hay `int xp = ent is Robot ? 30 : 40;` y en `IntentarEscapar` hay `int probabilidad = ent is Robot ? 70 : 50;`. Son verificaciones de tipo al estilo estructurado. Lo coherente con POO sería que `Personaje` tuviera propiedades virtuales

(`RecompensaXP`, `ProbabilidadEscape`) que cada derivada sobrescribe. Es una mejora que retomo en la sección V (caso de modificación).

II. Fundamentos de clases y objetos

2.1 Clases, objetos e instanciación

Una **clase** es la plantilla (qué datos y qué operaciones tiene). Un **objeto** es una instancia concreta, con su propio estado, creada con `new`.

```
// [NEXUS] Program.Main: una clase, tres objetos con estados distintos
RealidadDimensional realidadAsignada = new RealidadDimensional(nombreRealidad, 10, 6);
RealidadDimensional nexoFracturado   = new RealidadDimensional("Nexo Fracturado", 45, 12);
RealidadDimensional jardinEco        = new RealidadDimensional("Jardín de Eco", 5, 3);
```

Los tres usan la misma clase, pero `nexoFracturado` nace **Inestable** (45 ≥ `UMBRAL_INESTABLE` = 40) y los otros dos nacen **Estables**. Mismo molde, estado individual.

Otros ejemplos de instanciación en NEXUS:

- `GenerarEntidad()` crea `new Robot(...)` o `new Enemigo(...)` según un número al azar.
- `Cadete` crea su inventario al declararlo: `public Inventario Inventario { get; } = new Inventario();`.
- `Objeto.Crear(TipoObjeto)` es un **método de fábrica estático**: centraliza la creación de objetos.

Variables de referencia. En C#, las clases son tipos por referencia. En `Encuentro`, `entidadActual = ent;` no copia el objeto: ambas variables apuntan al **mismo** objeto. Por eso `Program` usa `ReferenceEquals(r, e.RealidadActual)` para saber si dos referencias señalan al mismo objeto (en `MostrarCatalogoRealidades` y `ViajarEntreRealidades`).

Clase estática. `Azar` es `static class`: no se puede instanciar, solo se usa `Azar.Rng`.

2.2 Atributos, campos, propiedades y métodos

- **Campo**: variable dentro de la clase. En NEXUS casi siempre es `private`.
- **Propiedad**: acceso controlado a un dato mediante `get` y `set`.
- **Método**: operación de la clase.
- **Atributo** (en teoría de POO): característica del objeto. En C# se implementa con campos o propiedades. No lo confundas con los *attributes* de C# (`[Serializable]`), que son otra cosa y no aparecen en NEXUS.

Forma	Ejemplo en NEXUS	Qué demuestra
Campo privado `readonly`	`private readonly string nombre;` (`RealidadDimensional`)	Se asigna solo en el constructor y no cambia después.
Campo privado mutable	`private int nivelInestabilidad;`	Cambia, pero solo mediante los métodos de la clase.
Constante	`public const int UMBRAL_CRITICA = 75;`	Valor fijo, común a toda la clase.
Propiedad completa de solo lectura	`public string Nombre { get { return nombre; } }`	Expone el campo sin permitir modificarlo.
Autopropiedad de solo lectura	`public string Nombre { get; }` (`Personaje`)	Solo se asigna en el constructor.

Forma	Ejemplo en NEXUS	Qué demuestra
Propiedad con setter restringido	`public int Vida { get; protected set; }`	Leer: cualquiera. Escribir: solo la clase y sus derivadas.
Propiedad calculada	`public bool EstaVivo { get { return Vida > 0; } }`, `Nivel` (`XP / 100 + 1`)	No guarda dato: lo calcula cada vez.
Evento	`public event Action<Explorador> AnomaliaDetectada;`	Permite que otros se suscriban.
Método de instancia	`RegistrarExploracion()`, `RecibirDano(int)`	Cambia o consulta el estado de un objeto concreto.
Método privado auxiliar	`CalcularEstado()`, `AjustarInestabilidad(int)`	Detalle interno que no debe usar nadie desde fuera.
Método estático	`Limitar(int)`, `Objeto.Crear(TipoObjeto)`	Pertenece a la clase, no a un objeto.

2.3 Constructores y sobrecarga

Un **constructor** inicializa el objeto y puede **rechazar** datos inválidos para que nunca exista un objeto en mal estado.

```
// [NEXUS] RealidadDimensional (constructor, resumido)
public RealidadDimensional(string nombre, int inestabilidadInicial, int factorRiesgo)
{
    if (string.IsNullOrWhiteSpace(nombre))
        throw new ArgumentException("Una realidad debe tener nombre.", nameof(nombre));
    if (factorRiesgo < 1 || factorRiesgo > 30)
        throw new ArgumentOutOfRangeException(nameof(factorRiesgo), "...");

    this.nombre = nombre.Trim();
    this.factorRiesgo = factorRiesgo;
    this.nivelInestabilidad = Limitar(inestabilidadInicial);
    this.vecesExploradas = 0;
    this.estado = CalcularEstado();   // el estado inicial se deduce de la inestabilidad
}
```

Observa dos decisiones: el nombre y el factor de riesgo inválidos **lanzan excepción**, mientras que una inestabilidad fuera de rango se **corrige** con `Limitar`.

Encadenamiento con la clase base [NEXUS]. Todas las derivadas llaman al constructor del padre:

```
protected Personaje(string nombre, int vida, int ataque) { ... }   // base
public Robot(string nombre) : base(nombre, 40, 6) { }              // fija vida 40, ataque 6
public Explorador(...) : base(nombre, 100, 12) { ... }             // llama a Cadete -> Personaje
```

El constructor de `Personaje` es `protected`: no se puede hacer `new Personaje(...)` (además es abstracta) y solo las derivadas pueden invocarlo.

Sobrecarga de constructores: NO está implementada en NEXUS. Sobrecargar es tener varios constructores (o métodos) con el mismo nombre y distinta lista de parámetros; el compilador elige uno según los argumentos. Lo más cercano en NEXUS son:

- **Parámetro opcional:** `Objeto(string nombre, TipoObjeto tipo, string icono, int cantidad = 1)` e `Inventario.Agregar(Objeto nuevo, bool ignorarLimite = false)`.
- **`params`:** `Mision(string nombre, int xp, params string[] objetivos)` acepta cualquier cantidad de objetivos.

Sí aparece sobrecarga *del framework* que NEXUS consume: `Math.Min` y `Console.WriteLine` tienen muchas versiones.

Ejemplo [COMPLEMENTARIO]: sobrecarga en `RealidadDimensional`:

```
// Constructor completo (el que ya existe)
public RealidadDimensional(string nombre, int inestabilidadInicial, int factorRiesgo) { ... }

// Sobrecarga: realidad «por defecto» con inestabilidad 0 y riesgo 5
public RealidadDimensional(string nombre) : this(nombre, 0, 5) { }
```

`new RealidadDimensional("Vacío")` ejecutaría el segundo constructor, que delega en el primero con `: this(...)`. Así la validación está escrita una sola vez.

Diferencia clave: un parámetro opcional es una sola firma con valores por defecto; una sobrecarga son varias firmas distintas. Para construcciones con comportamiento diferente, conviene sobrecargar.

2.4 Encapsulamiento y modificadores de acceso

Encapsular es ocultar el estado interno y permitir cambiarlo solo mediante operaciones que protegen las reglas del objeto.

Ejemplo completo: la inestabilidad de una realidad **jamás** sale del rango 0–100, y el estado **siempre** es coherente con ese valor, porque no hay forma de saltarse estos métodos:

```
// [NEXUS] RealidadDimensional
private void AjustarInestabilidad(int delta)
{
    nivelInestabilidad = Limitar(nivelInestabilidad + delta);   // nunca fuera de 0..100
    EstadoRealidad anterior = estado;
    estado = CalcularEstado();                                  // estado siempre recalculado

    if (estado != anterior && EstadoCambiado != null)
        EstadoCambiado(this, anterior, estado);                 // avisa solo si cambió
}
```

Desde fuera solo se puede llamar a `RegistrarExploracion()` o `Estabilizar(int)`. Lo mismo ocurre con `Explorador.Energia`: tiene `private set` y solo cambia con `AjustarEnergia`, que aplica `Limitar` (0–100).

Modificador	Significado	Ejemplo en NEXUS
`public`	Visible desde cualquier parte	`RealidadDimensional.Describir()`
`private`	Solo dentro de la propia clase	`CalcularEstado()`, `private int nivelInestabilidad`
`protected`	La clase y sus derivadas	Constructor de `Personaje`; `Vida { get; protected set; }`
`internal` (por defecto en clases sin modificador)	Todo el ensamblado	Todas las clases de NEXUS, porque no declaran modificador
`readonly` / `const`	No cambian tras la construcción / en compilación	`private readonly List<Objeto> objetos`, `Inventario.CAPACIDAD`

Fugas de encapsulamiento que puedes señalar (puntos críticos):

- `Personaje.Analizado { get; set; }` es público: cualquiera puede ponerlo en `true` sin pasar por la lógica de análisis.
- `Objeto.Cantidad { get; set; }`: desde fuera se podría poner `Cantidad = -5`.
- `Explorador.Conectado { get; set; }` se modifica libremente desde `Program`.

- `Mision.Objetivos` es `List<Objetivo>` de solo lectura *como referencia*, pero la lista se puede modificar desde fuera (`Objetivos.Add(...)`). Solución típica: exponer `IReadOnlyList<Objetivo>`.

2.5 Uso de `this` y `base`

`this`: el objeto actual [NEXUS]

1. **Desambiguar**: en el constructor de `RealidadDimensional`, el parámetro `nombre` oculta al campo `nombre`; `this.nombre = nombre.Trim();` indica «el campo de este objeto».
2. **Pasarse a sí mismo**: `EstadoCambiado(this, anterior, estado)` entrega a los suscriptores *esta* realidad; `AnomaliaDetectada(this)` en `Explorador` entrega *este* explorador. Gracias a eso `ManejarCambioEstadoRealidad` sabe cuál realidad cambió.
3. **Encadenar constructores** con `: this(...)`: no existe en NEXUS; ver el ejemplo complementario de 2.3.

`base`: la parte heredada [NEXUS]

1. **Constructor**: `: base(nombre, vida, ataque)` (en `Cadete`, `Robot`, `Enemigo`, y `: base(nombre, 100, 12)` en `Explorador`).
2. **Reutilizar el método del padre**: `Cadete.RecibirDano` primero aplica el escudo y luego delega el resto en `base.RecibirDano(dano)`:

```
// [NEXUS] Cadete
public override int RecibirDano(int dano)
{
    if (dano < 0) dano = 0;
    if (Escudo > 0)
    {
        int absorbido = Math.Min(Escudo, dano);
        Escudo -= absorbido;
        dano -= absorbido;
    }
    return base.RecibirDano(dano);   // descuenta de Vida el daño que sobró
}
```

Si en lugar de `base.RecibirDano(dano)` se escribiera `RecibirDano(dano)`, el método se llamaría a sí mismo sin fin y el programa terminaría con *StackOverflow*. `base` es lo que permite **extender** el comportamiento del padre en vez de reemplazarlo por completo.

III. Herencia, polimorfismo e interfaces

3.1 Clases base y derivadas: reutilización y especialización

Reutilizar es escribir algo una vez en la clase base y que todas las derivadas lo usen. **Especializar** es que cada derivada cambie o amplíe lo que hereda.

Clase	Reutiliza de su base	Especializa o añade [NEXUS]
`Personaje` (base)	—	`Nombre`, `VidaMax`, `Vida`, `Ataque`, `Analizado`, `EstaVivo`, `RecibirDano` (virtual), `Curar`
`Cadete`	Todo lo de `Personaje`	Escudo que absorbe daño, XP, `Nivel`, `BonoAtaque`, `CalcularAtaque`, `GanarXP`, `Inventario`
`Explorador`	Todo lo de `Cadete` y `Personaje`	Energía, estabilidad, peligro, fragmentos, `RealidadActual`, eventos

Clase	Reutiliza de su base	Especializa o añade [NEXUS]
`Robot`	Todo lo de `Personaje`	Blindaje: reduce 40 % del daño si no está analizado; reacción con pulso EMP
`Enemigo`	Todo lo de `Personaje`	Debilidad: +50 % de daño si está analizado; contraataque aleatorio

Ejemplos concretos de reutilización:

- `Curar(int)` está escrito **una sola vez** en `Personaje` y lo usan los cuatro tipos. `Cadete.GanarXP` lo llama al subir de nivel: `Curar(15)`.
- `EstaVivo` (`Vida > 0`) sirve igual para el Explorador, un Robot o un Enemigo: por eso `Encuentro` puede escribir `while (!escapo && ent.EstaVivo && e.EstaVivo)`.
- `Explorador` **no sobrescribe nada**: hereda `Describir` y `Reaccionar` de `Cadete`. Comprobable: `FinalizarJuego` llama a `e.Describir()` y el texto que imprime empieza con «Cadete ...», no con «Explorador ...».

3.2 Métodos `virtual` y `override`

`virtual` en la base significa «esta versión es la predeterminada, las derivadas pueden cambiarla». `override` en la derivada significa «reemplazo esa versión». Debe coincidir la firma exacta.

En NEXUS, el método virtual es `Personaje.RecibirDano` y tiene **cuatro versiones** relacionadas con `base`:

Clase	Qué hace antes de terminar en `Personaje.RecibirDano`
`Personaje` (base)	Corrige daño negativo a 0, calcula `real = Math.Min(dano, Vida)`, resta a `Vida`, devuelve `real`
`Cadete` (y `Explorador`, que la hereda)	El escudo absorbe primero; el sobrante pasa a `base`
`Robot`	Si **no** está analizado: `dano = dano * 6 / 10`
`Enemigo`	Si **sí** está analizado: `dano = dano * 3 / 2`

```
// [NEXUS] Personaje (versión base: lo común a todos)
public virtual int RecibirDano(int dano)
{
    if (dano < 0) dano = 0;
    int real = Math.Min(dano, Vida);
    Vida -= real;
    return real;
}
```

Reglas que suelen preguntar:

- Solo se puede hacer `override` de un método marcado `virtual`, `abstract` u `override`. `Curar` **no** es virtual: ninguna derivada puede reemplazarlo.
- El método devuelve el daño **realmente** aplicado (`real`), no el solicitado: por eso en el combate el mensaje muestra `-{real} vida`.
- Con `override` hay polimorfismo. Con `new` (ocultamiento) no lo habría: el método a ejecutar dependería del tipo de la variable, no del objeto. NEXUS no usa `new` de ese modo.

3.3 Clases y métodos `abstract`

```
// [NEXUS] Personaje
abstract class Personaje
{
    ...
    public abstract string Describir();
```

```
    public abstract string Reaccionar(Explorador cadete);
}
```

- Una **clase abstracta** no se puede instanciar: `new Personaje(...)` es un error de compilación. Sirve solo como base.
- Un **método abstracto** no tiene cuerpo. Toda clase derivada **no abstracta** está **obligada** a implementarlo con `override`. `Cadete`, `Robot` y `Enemigo` lo hacen. Si `Enemigo` olvidara `Reaccionar`, el proyecto no compilaría (error CS0534: no implementa un miembro abstracto heredado).
- `Cadete.Reaccionar` devuelve cadena vacía: está obligada a existir, aunque un Cadete no reaccione a nadie.

	`virtual`	`abstract`
¿Tiene cuerpo en la base?	Sí (comportamiento por defecto)	No
¿La derivada debe sobrescribirlo?	No, es opcional	Sí, es obligatorio
¿En qué clase puede estar?	Cualquier clase	Solo en una clase abstracta
En NEXUS	`RecibirDano`	`Describir`, `Reaccionar`

¿Por qué `Personaje` es abstracta? Un «personaje genérico» no tiene una descripción ni una reacción con sentido: solo los tipos concretos las conocen. Es un caso típico para un método abstracto.

3.4 Polimorfismo y resolución de llamadas a métodos

Polimorfismo: la misma llamada produce comportamientos distintos según el **tipo real** del objeto. Ejemplo central de NEXUS (`Program.Encuentro`):

```
Personaje ent = GenerarEntidad();      // tipo declarado: Personaje; tipo real: Robot o Enemigo
...
int real = ent.RecibirDano(dano);      // ¿qué RecibirDano se ejecuta?
Console.WriteLine(ent.Reaccionar(e));  // ¿qué Reaccionar?
```

`GenerarEntidad()` devuelve `Personaje` pero crea un `Robot` (40 % de las veces) o un `Enemigo`. El mismo código de `Encuentro` funciona para ambos.

Cómo se resuelve cada llamada:

1. **En compilación**, C# comprueba que el método exista en el **tipo declarado**. `ent.Analizado` compila; `ent.XP` **no** (XP pertenece a `Cadete`).
2. **En ejecución**, si el método es `virtual`, `abstract` u `override`, se ejecuta la versión de la **clase real** del objeto (la más derivada que lo sobrescribe).
3. Si el método **no** es virtual (por ejemplo `Curar`), se resuelve al compilar, según el tipo declarado.
4. `base.Metodo()` fuerza la versión del padre inmediato, sin importar el tipo real.

Resultado comprobable. Con un ataque de 20 puntos y sin análisis previo:

Objeto real	Se ejecuta	Daño aplicado
`Robot`	`Robot.RecibirDano`: 20 × 6 / 10 = 12	12
`Enemigo`	`Enemigo.RecibirDano`: sin cambio	20

Otros usos de polimorfismo en NEXUS:

- `Inventario.Utilizar(objeto, e, Personaje objetivo)` llama a `objetivo.Describir()`: imprime una cosa si es `Robot` y otra si es `Enemigo`.
- `Robot.Reaccionar` ejecuta `cadete.RecibirDano(Ataque + 2)`. El parámetro es un `Explorador`, pero se ejecuta `Cadete.RecibirDano`, que aplica primero el escudo y delega en `base`.

- `entidadActual` es un `Personaje` que puede contener cualquiera de los dos tipos.

Contra-ejemplo: `ent is Robot ? 30 : 40` (en `Encuentro`) pregunta por el tipo en vez de usar polimorfismo; es la parte menos «orientada a objetos» del combate.

3.5 Interfaces y contratos

Una **interfaz** es un contrato: declara **qué** debe poder hacer una clase (métodos, propiedades), no **cómo**. Una clase puede implementar varias interfaces, mientras que solo puede heredar de una clase.

En NEXUS no se declara ninguna interfaz propia. Sí se usan, sin que se vea, las del framework:

- `List<T>` implementa `IEnumerable<T>`, y LINQ está definido sobre esa interfaz. Por eso `objetos.Sum(...)` (una lista) y `conocidos.Any(...)` (un arreglo `string[]`) funcionan con la misma sintaxis.
- `StreamReader` implementa `IDisposable`, y es lo que permite el bloque `using (StreamReader lector = ...)` de `IrisConsola.EscucharMensajes`.

Ejemplo [COMPLEMENTARIO]: `IAtacable`. Hoy solo los `Personaje` pueden recibir daño. Con una interfaz, cualquier cosa (una barrera, una torreta) puede ser un objetivo:

```
public interface IAtacable
{
    bool EstaVivo { get; }
    int RecibirDano(int dano);
}

// Personaje ya tiene EstaVivo y RecibirDano públicos: basta con declararla.
abstract class Personaje : IAtacable { /* sin cambios */ }

// Una clase fuera de la jerarquía de personajes:
class BarreraEnergetica : IAtacable
{
    private int integridad = 50;
    public bool EstaVivo { get { return integridad > 0; } }
    public int RecibirDano(int dano)
    {
        int real = Math.Min(dano, integridad);
        integridad -= real;
        return real;
    }
}

// Uso polimórfico, sin importar la clase real:
List<IAtacable> blancos = new List<IAtacable> { new Robot("R-1"), new BarreraEnergetica() };
foreach (IAtacable b in blancos)
    Console.WriteLine(b.RecibirDano(10));
```

El código que ataca depende del **contrato** (`IAtacable`), no de una clase concreta. Esa es la ventaja principal: bajo acoplamiento. Adicionalmente, `Personaje.Reaccionar(Explorador)` podría recibir una interfaz en lugar del tipo concreto `Explorador`, para evitar la dependencia circular señalada en 1.3.

3.6 Herencia, composición e implementación de interfaces

	Herencia	Composición	Interfaz
Pregunta que responde	¿Es un tipo de...?	¿Tiene un...?	¿Sabe hacer...?
Acoplamiento	Alto (la derivada depende	Bajo	Bajo

	Herencia	Composición	Interfaz
	del padre)		
Comparte código	Sí	Sí, por delegación	No, solo la firma
Cantidad por clase	Una clase base	Cualquier cantidad de miembros	Varias interfaces
En NEXUS	`Robot` es un `Personaje`	`Cadete` tiene un `Inventario`; `Inventario` tiene `Objetos`	No hay (ejemplo: `IAtacable`)

Cómo decidir (criterio para justificar en el examen):

- Se usa **herencia** si se cumple el principio de sustitución: en cualquier lugar donde se espera un `Personaje` puede ir un `Robot` sin romper nada.
- Se usa **composición** si es una relación de «tiene»: un `Cadete` **no es** un inventario, **tiene** uno. Si se hubiera heredado `Cadete : Inventario`, un cadete tendría métodos como `Eliminar` como si fueran suyos, y sería ilógico.
- Se usa **interfaz** para que clases sin parentesco compartan un comportamiento sin compartir implementación.

IV. Colecciones y LINQ

4.1 Listas genéricas `List<T>`

Una `List<T>` es una colección de tamaño dinámico que solo acepta elementos de tipo `T`. La «T» genérica hace que sea **segura en tipos**: `List<Objeto>` no acepta un `string`, y no hace falta convertir al leer.

Lista	Dónde	Contiene
`private readonly List<Objeto> objetos`	`Inventario`	Los objetos del jugador
`public List<Objetivo> Objetivos`	`Mision`	Los 4 objetivos
`static List<RealidadDimensional> catalogoRealidades`	`Program`	Las 3 realidades

Operaciones de `List<T>` que NEXUS usa:

Operación	Ejemplo en NEXUS
`Add`	`objetos.Add(nuevo)` en `Inventario.Agregar`; `Objetivos.Add(new Objetivo(o))` en `Mision`
`Remove`	`objetos.Remove(objeto)` en `Inventario.Eliminar`
`Contains`	`objetos.Contains(objeto)` en `Utilizar` y `Eliminar`
`Count` (propiedad)	`objetos.Count` en `Consultar` y `Buscar`
Indexador `[i]`	`objetos[indice - 1]` en `Buscar`; `catalogoRealidades[eleccion - 1]` en `ViajarEntreRealidades`

Diferencia con un arreglo: el arreglo (`string[] pistas`, `char[,] mapa`) tiene tamaño fijo; la lista crece y se encoge. NEXUS usa arreglos donde el tamaño es fijo (pistas, plantillas del mapa) y listas donde cambia (inventario).

4.2 Manipulación y recorrido de objetos

Recorrido con `for` e índice [NEXUS], cuando se necesita el número de posición (`Inventario.Consultar`):

```
for (int i = 0; i < objetos.Count; i++)
    Console.WriteLine($"  {i + 1}. {objetos[i].Icono} {objetos[i].Nombre}
x{objetos[i].Cantidad}");
```

Recorrido con `foreach` [NEXUS], cuando solo importa cada elemento (`Mision.Mostrar`, y en `Main` la suscripción de eventos):

```
foreach (RealidadDimensional r in catalogoRealidades)
    r.EstadoCambiado += ManejarCambioEstadoRealidad;
```

Manipulación: la lógica de `Inventario.Agregar` (apilar cantidades en vez de duplicar entradas):

```
Objeto existente = objetos.FirstOrDefault(o => o.Nombre == nuevo.Nombre);
if (existente != null) existente.Cantidad += nuevo.Cantidad;   // ya hay uno igual: suma
else objetos.Add(nuevo);                                        // no hay: lo agrega
```

Dos células seguidas dejan **una** entrada «Célula de energía x2», no dos. Y `Total` cuenta unidades (suma de `Cantidad`), no entradas.

Cuidado, para el examen: dentro de un `foreach` sobre una lista **no se puede** agregar ni quitar elementos de esa misma lista (lanza `InvalidOperationException`). NEXUS lo evita: `Eliminar` se llama fuera de los recorridos.

4.3 Expresiones lambda

Una lambda es una función anónima en una línea: `parámetros => expresión`. Se lee «dado `o`, devuelve esto».

```
objetos.Sum(o => o.Cantidad)       // o es un Objeto; devuelve su Cantidad (int)
Objetivos.All(o => o.Completado)   // o es un Objetivo; devuelve un bool
```

- El tipo de `o` se **infiere** del tipo de la colección (`List<Objeto>` → `o` es `Objeto`).
- Una lambda puede **capturar variables locales** (se llama *clausura*). En `Buscar`, `objetos.FirstOrDefault(o => Sin(o.Nombre).Contains(entrada))` usa la variable `entrada` del método, que no es parte de la lambda.
- Es el argumento de los métodos LINQ, que esperan un delegado (`Func<T, TResult>`).

No confundir con los **delegados con nombre** de NEXUS: `Action<RealidadDimensional, EstadoRealidad, EstadoRealidad>` es el tipo del evento, y `ManejarCambioEstadoRealidad` es un método con nombre asignado con `+=`. Con una lambda sería [COMPLEMENTARIO]: `r.EstadoCambiado += (rd, ant, nue) => Console.WriteLine(rd.Nombre);`.

4.4 Consultas LINQ presentes en NEXUS

Operador	Dónde	Qué hace	Equivalente con bucle
`Sum`	`Inventario.Total`: `objetos.Sum(o => o.Cantidad)`	Suma un valor de cada elemento	Acumulador dentro de un `foreach`
`FirstOrDefault`	`Agregar` y `Buscar`	Primer elemento que cumple, o `null` si ninguno	`foreach` con `return` al encontrar
`Any`	`Buscar` (`conocidos.Any(...)`), `Mision.Marcar`	`true` si **algún** elemento cumple	`foreach` con `return true`
`All`	`Mision.Completada`	`true` si **todos** cumplen	`foreach` con `return false` al primer fallo
`Take`	`Mision.Marcar`: `Objetivos.Take(indice)`	Los primeros *n* elementos	Bucle `for` hasta *n*

Operador	Dónde	Qué hace	Equivalente con bucle
`Count` con condición	`Program.MostrarMenu`: `Objetivos.Count(o => o.Completado)`	Cuántos cumplen	Contador con `if`

Un ejemplo para analizar con calma (`Mision.Marcar`): el último objetivo (extracción) solo se puede cumplir si los anteriores ya están hechos.

```
if (indice == Objetivos.Count - 1 && Objetivos.Take(indice).Any(o => !o.Completado)) return false;
```

Para `indice = 3`, `Take(3)` toma los objetivos 0, 1 y 2; `Any(o => !o.Completado)` pregunta «¿falta alguno?». Si falta alguno, devuelve `false` y el objetivo de extracción no se marca. Por eso `ProgresoMision` muestra el aviso «aún hay objetivos pendientes».

Dos matices que se prestan a preguntas:

- `objetos.Count` (propiedad de `List`, sin paréntesis) no es lo mismo que `Count(o => ...)` (método LINQ, con condición).
- `All` sobre una colección **vacía** devuelve `true`. Una `Mision` creada sin objetivos estaría «completada» desde el inicio.

4.5 Filtrado, transformación y ordenamiento: `Where`, `Select`, `OrderBy` [COMPLEMENTARIO]

NEXUS **no usa** `Where`, `Select` ni `OrderBy`. Estos ejemplos están escritos con las clases reales del proyecto:

```
// Filtrar + ordenar + transformar, con el catálogo de Main
// (asignada: 10 -> Estable, Nexo Fracturado: 45 -> Inestable, Jardín de Eco: 5 -> Estable)
List<string> alertas = catalogoRealidades
    .Where(r => r.Estado != EstadoRealidad.Estable)           // filtrar
    .OrderByDescending(r => r.NivelInestabilidad)              // ordenar
    .Select(r => r.Nombre + ": " + r.NivelInestabilidad)       // transformar
    .ToList();
// Con los datos iniciales: ["Nexo Fracturado: 45"]
// Dentro de Inventario (la lista es privada, por eso va un método nuevo)
public List<string> NombresOrdenados()
{
    return objetos.OrderBy(o => o.Nombre).Select(o => o.Nombre).ToList();
}
// Inventario inicial -> ["Analizador", "Célula de energía", "Escudo"]

public int CantidadDeCelulas()
{
    return objetos.Where(o => o.Tipo == TipoObjeto.Celula).Sum(o => o.Cantidad);
}
// Inventario inicial -> 1
```

Operador	Papel	Ejemplo
`Where`	**Filtra**: deja los que cumplen la condición	Realidades que no están estables
`Select`	**Transforma**: convierte cada elemento en otra cosa	De `RealidadDimensional` a `string`
`OrderBy` / `OrderByDescending`	**Ordena** por una clave	Por inestabilidad, de mayor a menor
`ToList`	**Materializa**: ejecuta la consulta y devuelve una lista	—

Ejecución diferida: `Where`, `Select`, `OrderBy` y `Take` no calculan nada al escribirse; la consulta se ejecuta cuando se recorre o se llama a `ToList`. En cambio `Sum`, `Count`, `Any`, `All` y `FirstOrDefault` entregan su resultado de inmediato.

4.6 LINQ frente a recorridos tradicionales

La misma operación en ambos estilos (`Inventario.Total`):

```
// LINQ [NEXUS]: dice QUÉ se quiere
public int Total { get { return objetos.Sum(o => o.Cantidad); } }

// Recorrido tradicional [COMPLEMENTARIO]: dice CÓMO se hace
public int Total
{
    get
    {
        int suma = 0;
        foreach (Objeto o in objetos) suma += o.Cantidad;
        return suma;
    }
}
```

Y `Mision.Completada`:

```
// LINQ [NEXUS]
public bool Completada { get { return Objetivos.All(o => o.Completado); } }

// Tradicional [COMPLEMENTARIO]
foreach (Objetivo o in Objetivos)
    if (!o.Completado) return false;
return true;
```

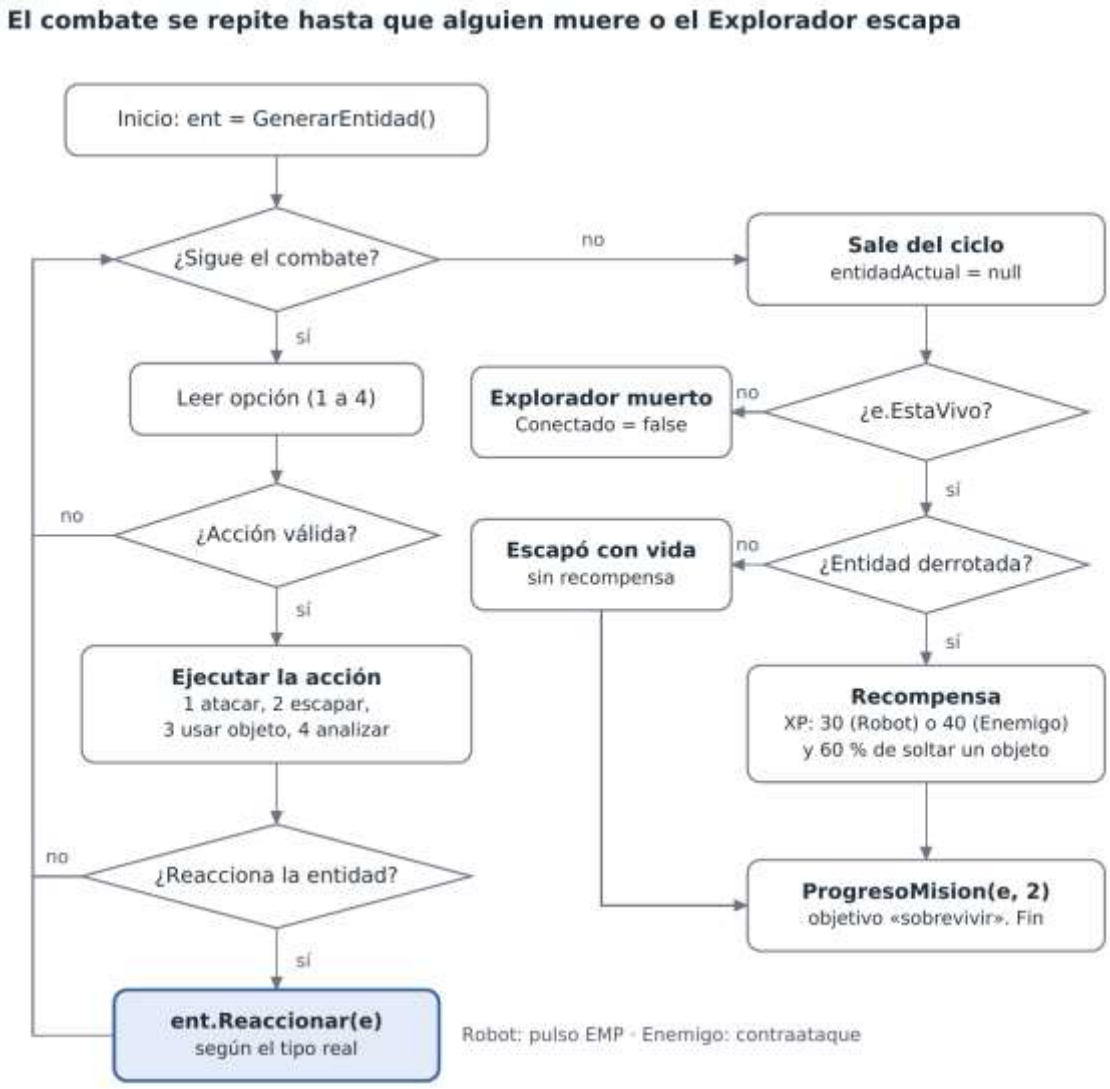
Criterio	LINQ	Bucle tradicional
Legibilidad	Alta cuando la consulta es corta: expresa la intención	Más líneas y variables auxiliares
Variables auxiliares	Ninguna	Acumuladores, contadores, banderas
Control fino (índice, `break`, modificar la lista)	Limitado	Total
Efectos secundarios (imprimir, cambiar estado)	No recomendable	Natural
Rendimiento	Pequeño costo extra (delegados e iteradores); irrelevante con listas de 10 elementos	Algo más directo

Por eso NEXUS combina ambos y lo hace bien: `Consultar` usa `for` porque necesita el número de posición y escribe en pantalla (efectos secundarios); `Total`, `Completada` y `Buscar` usan LINQ porque solo calculan un valor.

V. Análisis del funcionamiento

5.1 Flujograma: el encuentro de combate (`Program.Encuentro`)

El combate es el proceso donde más clases colaboran: `Explorador`, `Personaje` (en la práctica un `Robot` o un `Enemigo`), `Inventario` y `Mision`. El diagrama sigue el código tal como está escrito.



Flujograma del combate (Program.Encuentro)

Cómo leerlo:

- **Acción inválida** (opción distinta de 1 a 4, objeto inexistente, energía insuficiente para analizar): se lanza `OperacionInvalidaException`, se muestra el aviso y el ciclo continúa **sin gastar el turno** (`continue` dentro del `catch`). La opción 3 con el campo vacío tampoco gasta turno.
- **¿Reacciona la entidad?** Se cumple si no se escapó, la entidad sigue viva, el turno se gastó y además (`ent is Enemigo || op == "1" || op == "2"`). Es decir: el `Enemigo` contraataca siempre; el `Robot` solo cuando lo atacas o intentas huir. Si usas un objeto o analizas a un Robot, no reacciona.
- **Polimorfismo en el recuadro resaltado:** `ent.Reaccionar(e)` ejecuta el pulso EMP (`Robot`) o el contraataque (`Enemigo`) según el tipo real.
- **Salida del ciclo:** si el Explorador murió, se desconecta y se termina con `return` (no se llama a `ProgresoMision`). Si ganó, recibe XP y puede encontrar un objeto. Si la entidad sigue viva y el Explorador también, es que escapó. Tanto al vencer como al escapar se llama a `ProgresoMision(e, 2)`: el objetivo es *sobrevivir* al encuentro.

5.2 Seguimiento del estado de un objeto durante la ejecución

Objeto seguido: `nexoFracturado`, creado en `Main` con `new RealidadDimensional("Nexo Fracturado", 45, 12)`. Constantes: `UMBRAL_INESTABLE = 40`, `UMBRAL_CRITICA = 75`. Cada exploración suma `factorRiesgo = 12`. Cada `Estabilizar` resta la cantidad indicada. El estado se recalcula **siempre** con `>=`.

Paso	Instrucción	nivelInestabilidad	vecesExploradas	estado	¿Se dispara EstadoCambiado?
0	Constructor	45	0	Inestable	No (el constructor asigna `estado` directamente)
1	`RegistrarExploracion()`	57	1	Inestable	No
2	`RegistrarExploracion()`	69	2	Inestable	No
3	`RegistrarExploracion()`	81	3	**Critica**	Sí: Inestable → Critica
4	`Estabilizar(8)`	73	3	**Inestable**	Sí: Critica → Inestable (73 < 75)
5	`RegistrarExploracion()`	85	4	**Critica**	Sí: Inestable → Critica
6	`Estabilizar(0)`	85	4	Critica	No: `cantidad <= 0` hace `return`

Lo que se observa y se puede comprobar jugando:

- El evento solo se dispara cuando el estado **cambia**, no en cada ajuste (pasos 1, 2 y 6 no lo disparan).
- El paso 4 equivale a usar la opción *Recuperar energía* estando en esa realidad (`RecuperarEnergia` llama a `Estabilizar(8)`). El paso 3 aparece en pantalla como «cambió de estado: Inestable → Critica» y se envía el mismo texto a la consola IRIS.
- El estado condiciona los costos. `AplicarEfectosDeRealidad` lee el estado **después** de `RegistrarExploracion`: en el paso 1 cobra -5 energía y -5 estabilidad (Inestable); en el paso 3 cobra -10 y -12 (Critica).
- También cambia el costo de viajar hacia esa realidad: `ObtenerCostoViaje() = 10 + nivelInestabilidad / 10`. Con 45 vale 14; con 73 vale 17; con 85 vale 18 (la división entera descarta el decimal).

5.3 Predicción del resultado de un fragmento de código

```
Cadete c = new Cadete("Ana", 100, 10);
c.ActivarEscudo(30);
int a = c.RecibirDano(45);
int b = c.RecibirDano(500);
Console.WriteLine(a + " " + b + " " + c.Vida + " " + c.Escudo + " " + c.EstaVivo);
```

Predicción: `15 85 0 0 False`

Razonamiento paso a paso:

1. Se crea el cadete con `Vida = 100`. `ActivarEscudo(30)` deja `Escudo = 30`.
2. `RecibirDano(45)` ejecuta `Cadete.RecibirDano`: como `Escudo > 0`, `absorbido = Math.Min(30, 45) = 30`. El escudo queda en 0 y `dano` baja a 15. Entonces `base.RecibirDano(15)`: `real = Math.Min(15, 100) = 15`, `Vida = 85`, devuelve 15. Por eso `a = 15`.
3. `RecibirDano(500)`: el escudo ya es 0, no absorbe. `base.RecibirDano(500)`: `real = Math.Min(500, 85) = 85`, `Vida = 0`, devuelve 85. Por eso `b = 85`. La vida **no** queda negativa gracias a `Math.Min`.

4. `EstaVivo` es `Vida > 0`, o sea `False`. En C#, la concatenación imprime `False` con mayúscula.

5.4 ¿Qué ocurriría si se modifica una clase, un método o una condición?

Caso A: cambiar una condición/constante. Si `UMBRAL_CRITICA` pasa de 75 a 60, repite la tabla de 5.2: en el paso 2 el nivel (69) ya es ≥ 60, así que `nexoFracturado` pasa a **Critica una exploración antes** y cobra -10 energía y -12 estabilidad desde ese momento. Solo hay que editar una línea porque la constante es la **única fuente de verdad** (se usa únicamente en `CalcularEstado`). Es la ventaja de no escribir el número 75 en varios sitios.

Caso B: mover una decisión del `Program` a la clase. Hoy `IntentarEscapar` hace `ent is Robot ? 70 : 50`. Modificación coherente con POO:

```
// En Personaje
public virtual int ProbabilidadEscape { get { return 50; } }

// En Robot
public override int ProbabilidadEscape { get { return 70; } }

// En Program.IntentarEscapar
int probabilidad = ent.ProbabilidadEscape;   // sin preguntar por el tipo
```

El juego se comporta exactamente igual (Robot 70, Enemigo 50), pero un `Jefe` nuevo con probabilidad 20 solo necesitaría su propia clase: `Program` no se tocaría. Lo mismo vale para `ent is Robot ? 30 : 40` (XP).

Caso C: tabla de modificaciones y sus consecuencias.

Modificación	Consecuencia
Quitar `virtual` a `Personaje.RecibirDano`	No compila: los `override` de `Cadete`, `Robot` y `Enemigo` fallan (error CS0506: el miembro base no es virtual).
Quitar `override` en `Robot.RecibirDano`	Compila con una advertencia (CS0114, oculta el miembro heredado). Pero **se pierde el polimorfismo**: con `Personaje ent`, se ejecuta la versión base y el Robot recibiría 20 en vez de 12 (sin blindaje).
Cambiar `Vida { get; protected set; }` a `private set`	Sigue compilando: solo `Personaje` escribe en `Vida` (en `RecibirDano` y `Curar`). Es un indicio de que `protected` da más acceso del necesario (mínimo privilegio).
En `Mision.Marcar`, quitar `indice == Objetivos.Count - 1 &&`	Todos los objetivos pasarían a exigir cumplir los anteriores. Si el jugador sobrevive a un encuentro (objetivo 2) antes de llegar a la anomalía (objetivo 0), el objetivo **no se registraría**. La misión se volvería estrictamente secuencial y más difícil.

VI. Preparación para el examen

Cada pregunta incluye una **respuesta razonada** y los **puntos que debe tener una respuesta completa**. Intenta responder primero sin mirar.

Pregunta 1 · Comprensión de POO

`RealidadDimensional` guarda `nivelInestabilidad` y `estado` como campos `private`. (a) Explica qué principio de POO se aplica y qué problema evita. (b) Si `nivelInestabilidad` fuera `public`, describe una secuencia concreta de instrucciones que deje al objeto en un estado incoherente. (c) ¿Por qué `Mision.Objetivos` es un ejemplo de encapsulamiento incompleto?

Respuesta razonada

(a) Se aplica **encapsulamiento**: el estado interno solo cambia mediante métodos de la clase (`RegistrarExploracion`, `Estabilizar`), que mantienen las reglas del objeto: el nivel siempre entre 0 y 100 (`Limitar`) y el `estado` siempre consistente con el nivel (`CalcularEstado`). Además avisan por el evento solo cuando el estado cambia.

(b) Con el campo público, un cliente podría escribir `r.nivelInestabilidad = 90;`. El nivel quedaría en 90 pero `estado` seguiría en `Estable`. Nadie llamaría a `CalcularEstado`, no se dispararía `EstadoCambiado` y `ObtenerCostoExtraEnergia` devolvería 0 para una realidad que debería ser crítica. Incluso podría escribirse `-50` o `500` y romper el rango.

(c) `Objetivos` solo tiene `get`, así que no se puede reemplazar la lista, pero la referencia devuelta permite `m.Objetivos.Add(...)` o `Clear()` desde fuera. La clase pierde el control de su colección. Solución: guardarla privada y exponer `IReadOnlyList<Objetivo>`.

Puntos clave: nombrar encapsulamiento; relacionarlo con invariantes (rango y coherencia estado-nivel); dar un ejemplo concreto de ruptura; señalar la fuga de la lista.

Pregunta 2 · Análisis de clases y objetos

```
Inventario inv = new Inventario();
Objeto c = Objeto.Crear(TipoObjeto.Celula);
bool r1 = inv.Agregar(c);
bool r2 = inv.Agregar(Objeto.Crear(TipoObjeto.Celula));
bool r3 = inv.Agregar(new Objeto("Célula de energía", TipoObjeto.Celula, "*", 9));
bool r4 = inv.Agregar(Objeto.ModuloDesconocido(), true);
Console.WriteLine(r1 + " " + r2 + " " + r3 + " " + inv.Total);
```

(a) ¿Qué imprime? (b) ¿Cuántas entradas tiene la lista interna? (c) ¿Cuánto vale `c.Cantidad` al final y por qué?

Respuesta razonada

(a) Imprime `True True False 3`.

- `r1`: lista vacía, `Total + 1 = 1 ≤ 10`, no existe otro con ese nombre, se agrega `c`. Devuelve `true`.
- `r2`: `Total + 1 = 2 ≤ 10`. `FirstOrDefault` encuentra «Célula de energía» (es `c`) y se suma `Cantidad`: ahora vale 2. Devuelve `true`.
- `r3`: `Total + 9 = 11 > 10` y `ignorarLimite` es `false`: devuelve `false` y no modifica nada.
- `r4`: con `ignorarLimite = true` se agrega el módulo desconocido. `Total = 2 + 1 = 3`.

(b) Dos entradas: «Célula de energía» (x2) y «Módulo desconocido» (x1). `Total` suma **unidades** (3), no entradas (2).

(c) `c.Cantidad = 2`. `Objeto` es una clase (tipo por referencia): la lista guarda la misma referencia que `c`, no una copia. Cuando `r2` encuentra el objeto existente, modifica ese mismo objeto, y `c` lo refleja.

Puntos clave: diferenciar unidades de entradas; explicar la regla de apilado; entender referencias (objeto compartido); usar la condición del límite.

Pregunta 3 · Herencia y polimorfismo

```
List<Personaje> grupo = new List<Personaje>
{
    new Robot("R-1"),
    new Enemigo("E-1", 40, 10),
    new Cadete("C-1", 100, 10)
};
```

```
foreach (Personaje p in grupo)
{
    p.Analizado = true;
    Console.WriteLine(p.Nombre + ": " + p.RecibirDano(20) + " -> " + p.Vida);
}
```

(a) ¿Qué imprime? (b) ¿Qué imprimiría si se elimina la línea `p.Analizado = true;`? (c) Explica qué mecanismo decide qué `RecibirDano` se ejecuta.

Respuesta razonada

(a) La concatenación se evalúa de izquierda a derecha: primero se ejecuta `p.RecibirDano(20)` y **después** se lee `p.Vida`. Salida:

```
R-1: 20 -> 20
E-1: 30 -> 10
C-1: 20 -> 80
```

- `Robot` analizado: no aplica el blindaje, recibe 20, `Vida = 40 - 20 = 20`.
- `Enemigo` analizado: `20 * 3 / 2 = 30`, `Vida = 40 - 30 = 10`.
- `Cadete`: no tiene escudo (0) y `Analizado` no lo afecta. Recibe 20, `Vida = 80`.

(b) Sin analizar: `R-1: 12 -> 28` (20 × 6 / 10 = 12), `E-1: 20 -> 20` y `C-1: 20 -> 80`.

(c) **Despacho dinámico** (enlace tardío): `RecibirDano` es `virtual` en `Personaje` y las derivadas lo sobrescriben con `override`. La variable `p` es de tipo `Personaje`, pero en ejecución el CLR busca la versión de la **clase real** del objeto. Si `Robot` no tuviera `override`, se ejecutaría siempre la versión base.

Puntos clave: orden de evaluación; tipo declarado frente a tipo real; `virtual`/`override`; cálculo con enteros (`* 3 / 2`, `* 6 / 10`).

Pregunta 4 · Interfaces y diseño

Se quiere añadir una `Torreta` al combate: recibe daño y se destruye, pero no tiene vida máxima, no se cura ni gana nivel. Compara dos diseños: (A) `Torreta : Personaje`; (B) una interfaz `IAtacable` implementada por `Personaje` y por `Torreta`. ¿Cuál elegirías y por qué? ¿Qué cambiaría en `Program.Encuentro`?

Respuesta razonada

- **Diseño A (herencia):** `Torreta` heredaría `VidaMax`, `Ataque`, `Curar` y la obligación de implementar `Reaccionar(Explorador)` y `Describir`. Heredaría miembros que no tienen sentido para ella (curarse), lo que debilita la idea de que «es un `Personaje`» y la acopla a todos los cambios de la base.
- **Diseño B (interfaz):** el contrato mínimo es `bool EstaVivo { get; }` y `int RecibirDano(int dano)`. `Personaje` ya cumple ambos (basta declarar `: IAtacable`) y `Torreta` los implementa a su manera. No comparte código, pero tampoco arrastra lo que no necesita.
- **Elección:** B, porque la relación es «sabe recibir daño», no «es un personaje». Una clase solo puede heredar de una base, pero puede implementar varias interfaces (por ejemplo, `IAtacable` e `IDescribible`).
- **Cambio en `Encuentro`:** la variable `Personaje ent` pasaría a ser `IAtacable`, y el bucle `while (ent.EstaVivo ...)` y `ent.RecibirDano(dano)` seguirían funcionando. Pero hoy `Encuentro` también usa `Analizado`, `Describir`, `Reaccionar` y `ent is Robot`. Habría que moverlos a contratos adicionales (`IDescribible`, `IReaccionable`). Se ve que el diseño actual está acoplado a `Personaje`.

Puntos clave: distinguir «es un» de «sabe hacer»; limitación de una sola herencia; acoplamiento; contrato mínimo.

Pregunta 5 · Listas y LINQ

```
Mision m = new Mision("Prueba", 50, "A", "B", "C", "D");
bool x1 = m.Marcar(3);
bool x2 = m.Marcar(0);
bool x3 = m.Marcar(0);
m.Marcar(1);
m.Marcar(2);
bool x4 = m.Marcar(3);
Console.WriteLine(x1 + " " + x2 + " " + x3 + " " + x4 + " " + m.Completada);
```

(a) ¿Qué imprime? (b) Reescribe la condición de `Marcar` para el último objetivo con un `for`. (c) Escribe una consulta LINQ que devuelva, ordenadas alfabéticamente, las descripciones de los objetivos **pendientes**.

Respuesta razonada

(a) Imprime `False True False True True`.

- `Marcar(3)` al inicio: es el último índice y `Take(3).Any(o => !o.Completado)` es `true` (A, B y C están pendientes), así que devuelve `false`.
- `Marcar(0)`: no es el último, se marca y devuelve `true`.
- `Marcar(0)` otra vez: ya estaba completado, devuelve `false`.
- Tras marcar 1 y 2, `Marcar(3)`: `Take(3)` ya no tiene pendientes, se marca y devuelve `true`. Entonces `All(o => o.Completado)` es `true`.

(b)

```
if (indice == Objetivos.Count - 1)
{
    for (int i = 0; i < indice; i++)
        if (!Objetivos[i].Completado) return false;
}
```

(c) *[Complementario: NEXUS no usa estos operadores]*

```
List<string> pendientes = m.Objetivos
    .Where(o => !o.Completado)
    .OrderBy(o => o.Descripcion)
    .Select(o => o.Descripcion)
    .ToList();
```

Puntos clave: semántica de `Any` y `All`; saber que `Take(n)` solo toma los primeros *n*; orden `Where` → `OrderBy` → `Select`; `ToList` ejecuta la consulta.

Pregunta 6 · Predicción, corrección o modificación de código

Un compañero escribió esta clase para añadir un jefe al juego:

```
class Jefe : Personaje
{
    public Jefe(string nombre) : base(nombre, 200, 15) { }

    public int RecibirDano(int dano)
    {
        return base.RecibirDano(dano / 2);
    }
```

```
    public override string Describir()
    {
        return "Jefe " + Nombre;
    }
}
```

(a) Indica los errores o advertencias que da el compilador y por qué. (b) Corrige la clase. (c) Con la clase corregida, predice qué imprime `Personaje p = new Jefe("J"); Console.WriteLine(p.RecibirDano(21)); Console.WriteLine(p.Vida);`

Respuesta razonada

(a) Hay dos problemas:

1. **Error de compilación (CS0534):** `Jefe` no implementa el método abstracto `Reaccionar(Explorador)`, y `Personaje` lo exige.
2. **Advertencia (CS0114):** `RecibirDano` no lleva `override`, así que *oculta* el método heredado en lugar de sobrescribirlo. Aunque compilara, con `Personaje p = new Jefe(...)` se ejecutaría la versión base, sin el daño reducido: se perdería el polimorfismo.

(b)

```
class Jefe : Personaje
{
    public Jefe(string nombre) : base(nombre, 200, 15) { }

    public override int RecibirDano(int dano)
    {
        return base.RecibirDano(dano / 2);
    }

    public override string Describir() { return "Jefe " + Nombre; }

    public override string Reaccionar(Explorador cadete)
    {
        int dano = cadete.RecibirDano(Ataque);
        return "El jefe te golpea: -" + dano + " vida.";
    }
}
```

(c) Imprime `10` y luego `190`. `21 / 2` es una **división entera** y da 10. `base.RecibirDano(10)` aplica `Math.Min(10, 200) = 10`, devuelve 10 y deja `Vida = 190`. Si se hubiera mantenido la versión sin `override`, imprimiría 21 y 179.

Puntos clave: diferencia entre sobrescribir y ocultar; obligación de implementar miembros abstractos; división entera; uso de `base`.

Repaso final: diez ideas que debes poder explicar con tus palabras

1. Una clase es la plantilla; un objeto es una instancia con estado propio; las variables de clase guardan **referencias**.
2. Encapsular es proteger invariantes: campos privados y métodos que mantienen las reglas (`RealidadDimensional`).
3. `this` se refiere al objeto actual; `base` accede a la parte heredada (`base.RecibirDano`).

4. `virtual` ofrece una versión por defecto; `abstract` obliga a implementarla; `override` sobrescribe.
5. El tipo declarado decide qué se puede llamar; el tipo real decide qué versión se ejecuta.
6. NEXUS no tiene interfaces propias ni sobrecarga de constructores: preséntalos como ejemplos complementarios.
7. Herencia = «es un»; composición = «tiene un» (`Cadete` e `Inventario`); interfaz = contrato.
8. `List<T>` es una colección dinámica y de tipo seguro; `Add`, `Remove`, `Contains`, `Count` y el indexador son lo básico.
9. LINQ en NEXUS: `Sum`, `FirstOrDefault`, `Any`, `All`, `Take`, `Count`. `Where`, `Select`, `OrderBy` solo aparecen en ejemplos complementarios.
10. Un buen diseño minimiza las preguntas por tipo (`is Robot`) y deja que cada clase decida su comportamiento.

La inteligencia artificial como espejo

Diagnóstico de Evaluación de Aprendizaje — Misión NEXUS

Como evaluador de tu proceso de aprendizaje durante la misión **NEXUS: DE CÓDIGO A CONOCIMIENTO**, he realizado un análisis crítico y riguroso de nuestra interacción, enfocándome de manera estricta en tus intervenciones directas, preguntas, capturas de pantalla, decisiones operativas y aportes personales.

A continuación, se presenta el diagnóstico estructurado según los apartados solicitados:

---

1. Fortalezas demostradas

* **Ejecución procedimental y uso de herramientas:** Demostraste competencia técnica para operar GitHub Desktop y la plataforma web de GitHub. Esto se evidencia en tu capacidad para inicializar el repositorio, configurar el archivo `.gitignore` y añadir carpetas de proyecto de C#.
* **Capacidad de corrección operativa:** Cuando se te indicó que el repositorio se encontraba configurado como privado con un icono de candado (`JHONNGUTIERREZ / 🔒 JHONN`), interpretaste la retroalimentación y realizaste de forma exitosa los pasos visuales para cambiarlo a modo público hasta confirmar que la etiqueta cambiara a "Público".
* **Seguimiento metódico de instrucciones:** Mostraste constancia al verificar uno a uno los elementos de la lista de requerimientos (subida de archivos `.cs`, `.csproj`, `.sln` y creación del archivo `README.md`).

(Nota: En cuanto a conceptos teóricos o explicaciones de programación, no se registran aportes conceptuales explicados por ti en la conversación; tus aportes se centraron en la validación del proceso en plataforma).

---

2. Falencias observadas

* **Ausencia de argumentación teórica propia:** No emitiste explicaciones, definiciones ni razonamientos conceptuales sobre el código fuente analizado (`NexusTrainingSystem`). Delegaste la totalidad del desarrollo técnico y la redacción del manual al asistente.
* **Dependencia operativa en la resolución de dudas:** Ante cada paso, tu interacción se limitó a preguntas de validación externa ("esta bien o algo hice mal?", "donde puedo hacerlo?", "ahora que paso sigue?", "todos los pasos ya estan o falta algi ?") en lugar de realizar deducciones analíticas previas.
* **Falta de contraste crítico de contenidos:** No se observa evidencia de que hayas cuestionado, comparado o puesto a prueba los fragmentos de código o las respuestas teóricas provistas para verificar su comprensión profunda.

---

3. Diagnóstico del Hito 3

Evaluación de los conceptos del Hito 3 fundamentada exclusivamente en los aportes y evidencias generados por ti en la conversación:

Concepto	Clasificación	Justificación basada en evidencia
POO (Programación Orientada a Objetos)	Evidencia insuficiente	No aportaste definiciones, análisis de clases ni reflexiones sobre el paradigma de objetos.
Clases y Objetos	Evidencia insuficiente	Compartiste el código fuente del proyecto, pero no explicaste la instanciación ni la relación entre clases y objetos con tus propias palabras.
Constructores y Sobrecarga	Evidencia insuficiente	No existe registro de consultas o explicaciones orientadas a constructores o sobrecarga por parte tuya.
Encapsulamiento	Comprensión parcial (nivel operativo)	Demostraste manejo práctico del control de acceso a nivel de repositorios (visibilidad pública vs. privada). Sin embargo, sobre el encapsulamiento de código en C# (`private`, `public`, propiedades), hay evidencia insuficiente.
Herencia	Evidencia insuficiente	El código aportado contiene clases con herencia (`Cadete : Personaje`), pero no analizaste ni explicaste dicho mecanismo.
Polimorfismo	Evidencia insuficiente	No se registra ningún análisis o razonamiento tuyo sobre el uso de métodos `virtual` y `override`.
Interfaces	Evidencia insuficiente	No hay evidencias de comprensión ni preguntas relacionadas con contratos de interfaces.
**Listas genéricas `List<T>**`	Evidencia insuficiente	Aportaste el archivo con colecciones (`List<Objeto>`), pero no explicaste su funcionamiento ni su tipado genérico.
LINQ	Evidencia insuficiente	El código incluye consultas LINQ integradas, pero no formulaste razonamientos ni explicaciones sobre su ejecución.

---

4. Mi forma de aprender

* **Enfoque táctico y guiado:** Tu estrategia de aprendizaje durante esta misión ha sido de carácter puramente operativo e instrumental, apoyándote en validaciones paso a paso mediante capturas de pantalla para asegurar el cumplimiento de tareas de configuración técnica.
* **Reacción ante el error:** Presentaste una actitud receptiva y orientada a la acción al corregir la visibilidad del repositorio tras la alerta del asistente.
* **Área de mejora metodológica:** Para superar con éxito las evaluaciones teóricas y prácticas de Programación II, requieres transicionar de un rol de usuario que valida tareas procedimentales a un rol activo de análisis de código, donde expliques, critiques y resuelvas problemas de programación de forma autónoma.

---

5. Plan de estudio (Prioridades pre-examen y ejercicios prácticos)

Debes enfocar tu estudio independiente en los siguientes cinco temas críticos antes del examen H3, resolviendo los ejercicios propuestos **sin asistencia de IA**:

1. **Herencia y Polimorfismo**
* *Ejercicio práctico:* Toma la clase base `Personaje` y sus derivadas (`Robot`, `Enemigo`) del código que compartiste. Escribe en papel qué método se ejecuta en tiempo de ejecución si creas una variable de tipo `Personaje` que apunta a un `Robot`, y explica por qué.

2. **Encapsulamiento en C# (Modificadores y Propiedades)**
* *Ejercicio práctico:* Revisa la clase `RealidadDimensional`. Explica por qué sus campos principales son `private readonly` y se exponen mediante propiedades de solo lectura (`get`), y qué pasaría si fueran `public` sin restricciones.

3. **Colecciones y Listas Genéricas (`List<T>`)**
* *Ejercicio práctico:* Escribe un fragmento de código sencillo utilizando `List<string>` donde agregues tres elementos, busques uno mediante un método LINQ básico y elimines otro según su índice.

4. **Consultas LINQ y Expresiones Lambda**
* *Ejercicio práctico:* Toma la consulta `objetos.Sum(o => o.Cantidad)` de la clase `Inventario` y replícala usando un bucle `foreach` tradicional con una variable acumuladora.

5. **Manejo de Excepciones Personalizadas**
* *Ejercicio práctico:* Analiza la clase `OperacionInvalidaException : Exception` provista en tu código. Escribe un bloque `try-catch` donde lances esta excepción manualmente si un número ingresado por consola es menor a cero.

---

6. Conclusión

* **Conocimientos demostrados:** Has evidenciado sólidas habilidades instrumentales y procedimentales en la configuración de entornos de desarrollo, control de versiones con GitHub Desktop y publicación de repositorios públicos en la web.

* **Conocimientos que necesitan refuerzo:** Los fundamentos teóricos de la Programación Orientada a Objetos (POO), el diseño de clases, herencia, polimorfismo, interfaces y el uso de LINQ requieren un estudio conceptual profundo, ya que no fueron explicados ni analizados por ti durante el proceso.
* **Expectativa para el examen:** Sin un estudio teórico autónomo y la resolución práctica de código por cuenta propia, tendrás dificultades para resolver preguntas de análisis, predicción de código y diseño orientados a la comprensión que exige el examen H3. Debes ser capaz de explicar la lógica de las clases y resolver problemas de código sin apoyo externo.